

LAPORAN PRATIUM PEKAN 3 STRUKTUR DATA

“Stack”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2025

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “[]”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 8 April 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 17 April 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Pratikum.....	1
BAB II PEMBAHASAN	2
2.1 stackArray	2
2.2 StackPostFix.....	5
2.3 NilaiMaksimum	7
2.4 Siswa	9
BAB III KESIMPULAN.....	11
4.1 Kesimpulan	11
4.2 Saran.....	11
BAB IV DAFTAR PUSTAKA	12
Lampiran Tugas	14
1. Deskripsi Tugas.....	14
2. Aturan Penamaan	14
3. Struktur Program.....	14
4. Contoh Tampilan Output Program.....	15
5. Kriteria Penilaian	15
6. Deadline	15
Java (ADT) Class · Website_2511531009.....	16
Java (Main/Driver) Class · Browser_2511531009.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Stack (Tumpukan) adalah salah satu struktur data linear yang bekerja berdasarkan prinsip LIFO (Last In, First Out), dimana elemen yang terakhir dimasukkan akan menjadi elemen yang pertama kali dikeluarkan. Konsep ini dapat dianalogikan seperti tumpukan piring, dimana piring yang diletakkan paling akhir akan diambil pertama kali. Dalam Java, Stack diimplementasikan sebagai sebuah class yang merupakan turunan dari class Vector, yang mana class tersebut merupakan bagian dari Java Collections Framework pada paket java.util. Operasi utama pada Stack mencakup push() untuk menambahkan elemen ke puncak tumpukan, pop() untuk menghapus dan mengembalikan elemen teratas, serta peek() untuk melihat elemen teratas tanpa menghapusnya. Pemahaman terhadap Stack menjadi penting karena struktur data ini banyak digunakan dalam berbagai algoritma dan aplikasi nyata, seperti evaluasi ekspresi postfix, manajemen riwayat browser, dan mekanisme undo-redo pada berbagai aplikasi.

1.2 Tujuan

1. Mengetahui Stack dan memahami karakteristiknya, khususnya prinsip LIFO (*Last In, First Out*).
2. Menganalisis file Java yang mengimplementasikan struktur data Stack.

1.3 Manfaat Pratikum

1. Mampu menggunakan class Stack yang telah dipanggil ke dalam sebuah class Java.
2. Mampu menemukan dan memahami alur logika pada file Java yang memiliki class struktur data Stack.

BAB II PEMBAHASAN

2.1 stackArray

```
1 package pekan3_2511531009;
2
3 public class stackArray_2511531009 {
4     static final int MAX_1009 = 1000;
5     int top_1009;
6     int a_1009[] = new int[MAX_1009];
7     boolean isEmpty_1009() {
8         return (top_1009 < 0);
9     }
10
11     stackArray_2511531009() {
12         top_1009 = -1;
13     }
14
15     boolean push_1009(int x) {
16         if (top_1009 >= (MAX_1009 - 1)) {
17             System.out.println("Stack overflow");
18             return false;
19         } else {
20             a_1009[++top_1009] = x;
21             System.out.printf("%d dimasukkan dalam stack%n", x);
22             return true;
23         }
24     }
25
26     int pop_1009() {
27         if (top_1009 < 0) {
28             System.out.println("Stack Underflow");
29             return 0;
30         } else {
31             int x = a_1009[top_1009--];
32             return x;
33         }
34     }
35 }
```

Pada file ini kita akan diajak untuk mengenal Stack berbasis array pada Java. Untuk menggunakan Stack kita harus memanggil class tersebut dengan cara sebagai berikut:

`Stack<dT> a = new Stack<dT>();`

Dimana dT merupakan Data Type dari Stack tersebut dan a merupakan nama variabel Stack yang kita buat.

Beberapa method umum pada Stack adalah push(), pop(), peek(), isEmpty(), dan search().

Pada kode tersebut kita menginisialisasi sebuah Stack bertipe Integer. Pada proses push, kita memasukkan beberapa elemen ke dalam Stack sehingga elemen yang terakhir dimasukkan berada di posisi paling atas (puncak tumpukan). Method pop() digunakan untuk mengeluarkan elemen yang berada di puncak Stack, sedangkan peek() digunakan untuk melihat elemen teratas tanpa mengeluarkannya dari Stack. Method isEmpty() mengembalikan nilai boolean yang menyatakan apakah Stack tersebut kosong atau tidak, dan search() mengembalikan posisi berbasis satu dari elemen yang dicari, dimana nilai satu menunjukkan posisi puncak Stack. Output dari kode tersebut menunjukkan urutan elemen sesuai dengan prinsip LIFO, dimana elemen yang terakhir di-push akan menjadi elemen yang pertama kali di-pop.

```
36●    int peek_1009() {
37●        if (top_1009 < 0) {
38            System.out.println("Stack Underflow");
39            return 0;
40●        } else {
41            int x = a_1009[top_1009];
42            return x;
43        }
44    }
45
46●    void print_1009() {
47●        for(int i_1009 = top_1009; i_1009 > -1 ; i_1009--) {
48            System.out.print(" " + a_1009[i_1009]);
49        }
50    }
51 }
```

```
1  package pekan3_2511531009;
2
3  public class stackArrayDriver_2511531009 {
4
5●    public static void main(String[] args) {
6        stackArray_2511531009 s = new stackArray_2511531009();
7        s.push_1009(10);
8        s.push_1009(20);
9        s.push_1009(30);
10       System.out.println(s.pop_1009() + " dikeluarkan dari stack");
11       System.out.println("Elemen teratas adalah : " + s.peek_1009());
12       System.out.print("Element pada stack : ");
13       s.print_1009();
14   }
15
16 }
```

```
10 dimasukkan dalam stack  
20 dimasukkan dalam stack  
30 dimasukkan dalam stack  
30 dikeluarkan dari stack  
Elemen teratas adalah : 20  
Element pada stack : 20 10
```

2.2 StackPostFix

```
1 package pekan3_2511531009;
2 import java.util.*;
3 public class StackPostFix_2511531009 {
4     public static int postfixEvaluate_1009(String expression)
5     {
6         Stack<Integer> s_1009 = new Stack<Integer>();
7         Scanner ip_1009 = new Scanner(expression);
8
9         while (ip_1009.hasNext() ) {
10             if (ip_1009.hasNextInt()) {
11                 s_1009.push(ip_1009.nextInt());
12             } else {
13                 String op = ip_1009.next();
14                 int oprnd2 = s_1009.pop();
15                 int oprnd1 = s_1009.pop();
16
17                 if (op.equals("+")) {
18                     s_1009.push(oprnd1 + oprnd2);
19                 }
20                 else if (op.equals("-")) {
21                     s_1009.push(oprnd1 - oprnd2);
22                 }
23                 else if (op.equals("*")) {
24                     s_1009.push(oprnd1 * oprnd2);
25                 }
26                 else {
27                     s_1009.push(oprnd1 / oprnd2);
28                 }
29             }
30         }
31         ip_1009.close();
32         return s_1009.pop();
33     }
34 }
```

Pada file ini kita akan mempelajari implementasi Stack untuk mengevaluasi ekspresi postfix (notasi RPN / *Reverse Polish Notation*). Ekspresi postfix adalah ekspresi matematika dimana operator ditempatkan setelah operand-operandnya, berbeda dengan notasi infix yang biasa kita kenal.

Cara kerja evaluasi postfix menggunakan Stack adalah sebagai berikut: program akan menelusuri setiap karakter pada string ekspresi yang diberikan. Jika karakter tersebut merupakan angka (operand), maka karakter tersebut akan di-push ke dalam Stack. Jika karakter tersebut merupakan operator (+, -, *, /), maka program akan melakukan pop sebanyak dua kali untuk mengambil dua operand teratas, kemudian melakukan operasi aritmatika sesuai operator yang ditemukan, dan hasilnya di-push kembali ke dalam

Stack. Setelah seluruh karakter selesai ditelusuri, nilai yang tersisa di puncak Stack merupakan hasil akhir dari evaluasi ekspresi postfix tersebut. Pendekatan ini mendemonstrasikan salah satu kegunaan nyata dari struktur data Stack dalam pemrosesan ekspresi matematika.

Berikut adalah beberapa contoh dari hasil pemograman tersebut.

```
34
35 public static void main(String[] args) {
36     System.out.println("hasil postfix = " + postfixEvaluate_1009("5 2 4 * + 7 -"));
37 }
38 }
```

```
<terminated> StackPostFix_25
hasil postfix = 6
```

```
34
35 public static void main(String[] args) {
36     System.out.println("hasil postfix = " + postfixEvaluate_1009("5 2 5 * + 7 -"));
37 }
38 }
```

```
<terminated> StackPostFix_2
hasil postfix = 8
```

2.3 NilaiMaksimum

```
1 package pekan3_2511531009;
2
3 import java.util.Stack;
4
5 public class NilaiMaksimum {
6
7     public static int max(Stack<Integer> s_1009) {
8         Stack<Integer> backup_1009= new Stack<Integer>();
9
10        int maxValue_1009 = s_1009.pop();
11        backup_1009.push(maxValue_1009);
12
13        while (!s_1009.isEmpty()) {
14            int next = s_1009.pop();
15            backup_1009.push(next);
16            maxValue_1009 = Math.max(maxValue_1009, next);
17        }
18
19        while (!backup_1009.isEmpty()) {
20            s_1009.push(backup_1009.pop());
21        }
22        return maxValue_1009;
23    }
24
25    public static void main(String[] args) {
26        Stack<Integer> s_1009 = new Stack<Integer>();
27        s_1009.push(70);
28        s_1009.push(12);
29        s_1009.push(20);
30        System.out.println("isis stack " + s_1009);
31        System.out.println("Stack Teratas " + s_1009.peek());
32        System.out.println("Nilai maximum " + max(s_1009));
33    }
34
35 }
36
37 }
```

Pada file ini kita akan mempelajari implementasi Stack untuk mencari nilai maksimum dari sekumpulan elemen yang tersimpan di dalamnya. Pendekatan ini memanfaatkan method peek() dan iterasi pada Stack untuk membandingkan seluruh elemen tanpa menghapusnya dari tumpukan.

Program menginisialisasi Stack bertipe Integer kemudian memasukkan beberapa elemen melalui method push(). Untuk menemukan nilai maksimum, program menelusuri seluruh elemen pada Stack dengan memanfaatkan perulangan. Pada setiap iterasi, nilai elemen yang sedang ditinjau dibandingkan dengan nilai maksimum sementara yang telah tersimpan sebelumnya. Jika nilai elemen tersebut lebih besar dari nilai maksimum sementara, maka nilai maksimum sementara akan diperbarui. Proses ini berlanjut hingga seluruh elemen pada Stack telah ditelusuri, sehingga pada akhir iterasi variabel tersebut menyimpan nilai maksimum

dari seluruh elemen Stack. Output dari kode ini menunjukkan nilai terbesar dari elemen-elemen yang telah di-push ke dalam Stack sebelumnya.

Berikut adalah hasil output yang diberikan :

```
isi stack [70, 12, 20]  
Stack Teratas 20  
Nilai maximum 70
```

2.4 Siswa

```
1 package pekan3_2511531009;
2
3 import java.util.ArrayList;
4
5 class Siswa_2511531009 {
6
7     String nama;
8     int nim;
9
10    public Siswa_2511531009(String nama, int nim) {
11        this.nama = nama;
12        this.nim = nim;
13    }
14
15    @Override
16    public String toString() {
17        return "NIM : " + nim +
18            ", Nama : " + nama;
19    }
20 }
21
22 public class SiswaStack_2511531009 {
23     private ArrayList<Siswa_2511531009> stack_1009;
24
25     public SiswaStack_2511531009() {
26         stack_1009 = new ArrayList<>();
27     }
28
29     public void push_1009(Siswa_2511531009 mhs) {
30         stack_1009.add(mhs);
31     }
32
33     public Siswa_2511531009 pop_1009() {
34         if (!stack_1009.isEmpty()) {
35             return stack_1009.remove(stack_1009.size() - 1);
36         }
37         return null;
38     }
39 }
```

Pada file ini kita akan diajak untuk mengimplementasikan Stack dengan menggunakan objek kustom sebagai tipe datanya, yaitu objek dari class Siswa. Berbeda dengan contoh sebelumnya yang menggunakan tipe data primitif atau wrapper, pada file ini Stack menyimpan objek yang memiliki beberapa instance variable sekaligus.

Class Siswa menginisialisasi variabel-variabel penting seperti nama dan nilai sebagai atribut dari objek tersebut. Terdapat pula method toString() yang di-override agar representasi objek dapat ditampilkan sebagai teks yang mudah dibaca saat dicetak.

Pada class Driver, Stack diinisialisasi dengan menggunakan objek Siswa sebagai tipe data dari elemennya, yakni Stack<Siswa>. Beberapa objek Siswa kemudian dibuat dan di-push ke dalam Stack. Proses peek() digunakan untuk menampilkan objek yang berada di puncak Stack tanpa

menghapusnya, sedangkan `pop()` digunakan untuk mengeluarkan objek teratas dari tumpukan. Output dari program ini mendemonstrasikan bahwa Stack tidak hanya dapat menyimpan tipe data primitif, melainkan juga dapat menyimpan objek dari class kustom yang kita definisikan sendiri, sehingga membuktikan fleksibilitas Stack sebagai struktur data dalam Java.

```
41 public Siswa_2511531009 peek_1009() {
42     if (!stack_1009.isEmpty()) {
43         return stack_1009.get(stack_1009.size() - 1);
44     }
45     return null;
46 }
47
48 public boolean isEmpty() {
49     return stack_1009.isEmpty();
50 }
51
52 public void tampilkanSiswa() {
53     for (int i = stack_1009.size() - 1; i >= 0; i--) {
54         System.out.println(stack_1009.get(i));
55     }
56 }
57
58 public static void main(String[] args) {
59     SiswaStack_2511531009 studentStack = new SiswaStack_2511531009();
60
61     Siswa_2511531009 mhs1_1009 = new Siswa_2511531009("Ali", 1);
62     Siswa_2511531009 mhs2_1009 = new Siswa_2511531009("Bobby", 2);
63     Siswa_2511531009 mhs3_1009 = new Siswa_2511531009("Charles", 3);
64
65     studentStack.push_1009(mhs1_1009);
66     studentStack.push_1009(mhs2_1009);
67     studentStack.push_1009(mhs3_1009);
68
69     System.out.println("Siswa di dalam stack");
70     studentStack.tampilkanSiswa();
71
72     System.out.println("siswa teratas " + studentStack.peek_1009());
73     System.out.println("mengeluarkan siswa teratas dari stack : " + studentStack.pop_1009());
74     System.out.println("daftar siswa setelah di pop_1009 : ");
75     studentStack.tampilkanSiswa();
76 }
77
78
79 }
```

```
Siswa di dalam stack
NIM : 3, Nama : Charles
NIM : 2, Nama : Bobby
NIM : 1, Nama : Ali
siswa teratas NIM : 3, Nama : Charles
mengeluarkan siswa teratas dari stack : NIM : 3, Nama : Charles
daftar siswa setelah di pop_1009 :
NIM : 2, Nama : Bobby
NIM : 1, Nama : Ali
```

BAB III KESIMPULAN

4.1 Kesimpulan

Dalam pembelajaran praktikum ini dapat disimpulkan bahwa .dapat disimpulkan bahwa Stack merupakan sebuah class implementasi dari struktur data tumpukan yang bekerja dengan prinsip LIFO (*Last In, First Out*). Kita juga memahami keunggulan Stack dalam berbagai kasus penggunaan nyata, seperti evaluasi ekspresi postfix dan pencarian nilai maksimum, dimana prinsip LIFO yang dimilikinya menjadi kunci dalam menyelesaikan permasalahan tersebut secara efisien.

4.2 Saran

Berdasarkan hasil praktikum ini, ada beberapa saran yang bisa diterapkan untuk ke depannya. Seperti, mahasiswa sebaiknya diarahkan untuk lebih memahami perbedaan penggunaan Stack dibandingkan dengan ArrayList yang telah dipelajari pada pekan sebelumnya, sehingga mereka dapat menentukan struktur data yang paling sesuai untuk digunakan pada kasus-kasus tertentu.

BAB IV

DAFTAR PUSTAKA

- [1] Pengguna Stack Overflow, "How to read all elements of Stack without popping them?," *Stack Overflow*, 29 Jan. 2014. [Daring]. Tersedia pada: <https://stackoverflow.com/questions/21470861/how-to-read-all-elements-of-stack-without-popping-them>. [Diakses: 20 Apr. 2026].
- [2] GeeksforGeeks, "Stack Class in Java," *GeeksforGeeks*, 10 Apr. 2026. [Daring]. Tersedia pada: <https://www.geeksforgeeks.org/java/stack-class-in-java/>. [Diakses: 20 Apr. 2026].
- [3] Oracle, "Class Stack<E> — Java™ Platform Standard Ed. 8," *Oracle Java SE 8 API Documentation*. [Daring]. Tersedia pada: <https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>. [Diakses: 20 Apr. 2026].

LAPORAN TUGAS PEKAN 3 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2025

Lampiran Tugas

Topik : Implementasi List of Objects menggunakan ArrayList

Tema : Sistem Manajemen Playlist Musik

1. Deskripsi Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol “Back” ditekan.

2. Aturan Penamaan

- 1) Nama Kelas : concat sufiks “_” + NIM
Contoh : Website_2311531009 dan Browser_2311531009
- 2) Nama Variabel Atribut : concat sufiks “_” + NIM mod 10^4
Contoh : String judul_1009, String url_1009

3. Struktur Program

- 1) Kelas ADT Website (Website_2511531009)
Atribut :
 - Judul Website
 - URL WebsiteMethod :
 - Constructor untuk menginisialisasi atribut.
 - Selektor (Getter) untuk setiap atribut.
 - Mutator (Setter) untuk setiap atribut.
- 2) Kelas Driver (Browser_2511531009)
 - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan.
 - Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus.
 - Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan.
 - Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan `.isEmpty()`

4. Contoh Tampilan Output Program

=== Browser History NIM : 2511531009 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar

Pilihan : 1

Masukkan Judul : SSO UNAND

Masukkan URL : <https://sso.unand.ac.id/login>

Berhasil mengunjungi halaman!

5. Kriteria Penilaian

- 1) Ketepatan Struktur

Penggunaan Stack<NamaKelasADT> dan pemahaman LIFO yang benar.

- 2) Kesesuaian Aturan Penamaan :

Nama kelas (NIM lengkap) dan variabel (4 digit NIM).

- 3) Penanganan Kondisi :

Program berjalan tanpa error dan menu berfungsi baik.

- 4) Kerapihan Kode :

Penulisan indentasi dan pemberian komentar pada bagian Stack.

6. Deadline

Tugas dikumpulkan paling lambat pada hari Minggu, tanggal 19 April 2026, pukul 23.59 WIB.

Java (ADT) Class · Website 2511531009

```
1 package pekan3_2511531009;
2
3 public class Website_2511531009 {
4     String judulWebsite_1009;
5     String url_1009;
6
7     // Constructors
8     Website_2511531009(String jW_1009, String u_1009) {
9         this.judulWebsite_1009 = jW_1009;
10        this.url_1009 = u_1009;
11    }
12
13    // Selectors (Getter)
14    public String getJudulWebsite_1009() {
15        return judulWebsite_1009;
16    }
17
18    public String getURL_1009() {
19        return url_1009;
20    }
21
22    // Mutator (Setter)
23    public void setJudulLagu_1009(String jW_1009) {
24        this.judulWebsite_1009 = jW_1009;
25    }
26
27    public void setPenyanyi_1009(String u_1009) {
28        this.url_1009 = u_1009;
29    }
30
31    @Override
32    public String toString() {
33        return "Judul Website : " + judulWebsite_1009 +
34            ", URL Website : " + url_1009;
35    }
36 }
```

Java (Main/Driver) Class · Browser 2511531009

```

1 package pekan3_2511531009;
2 import java.util.*;
3 public class Browser_2511531009 {
4
5     // Tampilkan Menu
6     public static void tampilkanMenu_1009() {
7         System.out.println("=== Browser History NIM : 2511531009 ===");
8         System.out.println("1. Kunjungi Website (Push)");
9         System.out.println("2. Tombol Back (Pop)");
10        System.out.println("3. Lihat Halaman Aktif (Peek)");
11        System.out.println("4. Cek Status History");
12        System.out.println("5. Keluar");
13    }
14
15    // 1. Kunjungi Website (Push)
16    public static void kunjungiWebsite_1009(Stack<Website_2511531009> stack_1009, Scanner sc_1009) {
17        String[] labels_1009 = {"Masukkan Judul Website", "Masukkan URL Website"};
18        int max_1009 = Arrays.stream(labels_1009).mapToInt(String::length).max().getAsInt(); // sorry bro
19
20        System.out.printf("%-" + max_1009 + "s : ", labels_1009[0]);
21        String jw_1009 = sc_1009.nextLine();
22
23        System.out.printf("%-" + max_1009 + "s : ", labels_1009[1]);
24        String u_1009 = sc_1009.nextLine();
25
26        stack_1009.push(new Website_2511531009(jw_1009, u_1009)); // menambahkan elemen ke puncak stack
27        System.out.println("\nBerhasil mengunjungi halaman!\n");
28    }
29
30    // 2. Tombol Back (Pop)
31    public static void tombolBack_1009(Stack<Website_2511531009> stack_1009) {
32        if (!stack_1009.isEmpty()) { // cek stack
33            Website_2511531009 backpage_1009 = stack_1009.pop(); // backpage = popped element
34            System.out.println("\nKembali");
35            System.out.println("Halaman " + backpage_1009.getJudulWebsite_1009() +
36                " berhasil dihapus dari history\n");
37        } else {
38            System.out.println("\nBelum ada halaman yang dikunjungi");
39        }
40    }
41
42    // 3. Lihat Halaman Saat Ini (Peek)
43    public static void lihatHalamanSaatIni_1009(Stack<Website_2511531009> stack_1009) {
44        if (!stack_1009.isEmpty()) { // cek stack
45            Website_2511531009 lastPage_1009 = stack_1009.peek(); // intip halaman terakhir
46            System.out.println("Halaman terakhir dikunjungi adalah : " +
47                lastPage_1009.getJudulWebsite_1009() + "\n");
48        } else {
49            System.out.println("\nBelum ada halaman yang dikunjungi\n");
50        }
51    }
52
53    // 4. Cek Status History
54    public static void cekStatusHistory_1009(Stack<Website_2511531009> stack_1009) {
55        if (!stack_1009.isEmpty()) {
56            System.out.println("\nHistory saat ini (" + stack_1009.size() + " halaman):");
57            // Print dari atas to bawah
58            for (int i = stack_1009.size() - 1; i >= 0; i--) {
59                System.out.println("  - " + stack_1009.get(i).getJudulWebsite_1009());
60            }
61            System.out.println();
62        } else {
63            System.out.println("\nBelum ada halaman yang dikunjungi");
64        }
65    }

```

```

67 public static void main(String[] args) {
68     Stack<Website_2511531009> webStack_1009 = new Stack<>();
69     Scanner i_1009 = new Scanner(System.in);
70     int choice_1009;
71     do {
72         tampilkanMenu_1009();
73         System.out.print("Pilihan : ");
74         choice_1009 = i_1009.nextInt();
75         i_1009.nextLine();
76         System.out.println();
77
78         switch (choice_1009) {
79             case 1 :
80                 kunjungiWebsite_1009(webStack_1009, i_1009);
81                 break;
82             case 2 :
83                 tombolBack_1009(webStack_1009);
84                 break;
85             case 3 :
86                 lihatHalamanSaatIni_1009(webStack_1009);
87                 break;
88             case 4 :
89                 cekStatusHistory_1009(webStack_1009);
90                 break;
91             case 5 :
92                 System.out.println("Keluar dari program");
93                 break;
94             default:
95                 System.out.println("Pilihan tidak valid.\n");
96         }
97     } while (choice_1009 != 5);
98     i_1009.close();
99
100 }
101
102 }

```

```

=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 1

Masukkan Judul Website : Facebook
Masukkan URL Website : facebook.com

```

```

=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 4

History saat ini (3 halaman):
- Youtube
- Instagram
- Facebook

```

```
=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 3
|
Halaman terkahir dikunjungi adalah : Youtube
```

```
=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 2

Kembali
Halaman Youtube berhasil dihapus dari history

=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 4
|

History saat ini (2 halaman):
- Instagram
- Facebook
```

```
=== Browser History NIM : 2511531009 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan : 5

Keluar dari program
```